CODING ASSIGNMENT 1

NAME: D.Megha vardhan

ROLL NO: 2311CS040180

SUBJECT: Agentic AI

1. LLM Workflow (5 Marks): Develop a Python program that accepts user input and generates a response using an LLM (OpenAI/Gemini/Ollama).

Objective:

To accept user input and generate a response using a Large Language Model (LLM).

Python code:

```
from google import genai

client = genai.Client(
    api_key="AQ.Ab8RN6KjqSW6dV56BW2aMsfa8dE8Yp8J9v1x7ooqUeUsqF1KOg"
)

question = input("Enter your question: ")

# Call Gemini model
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=question
)

print("\nResponse:")
print(response.text)
```

Input:

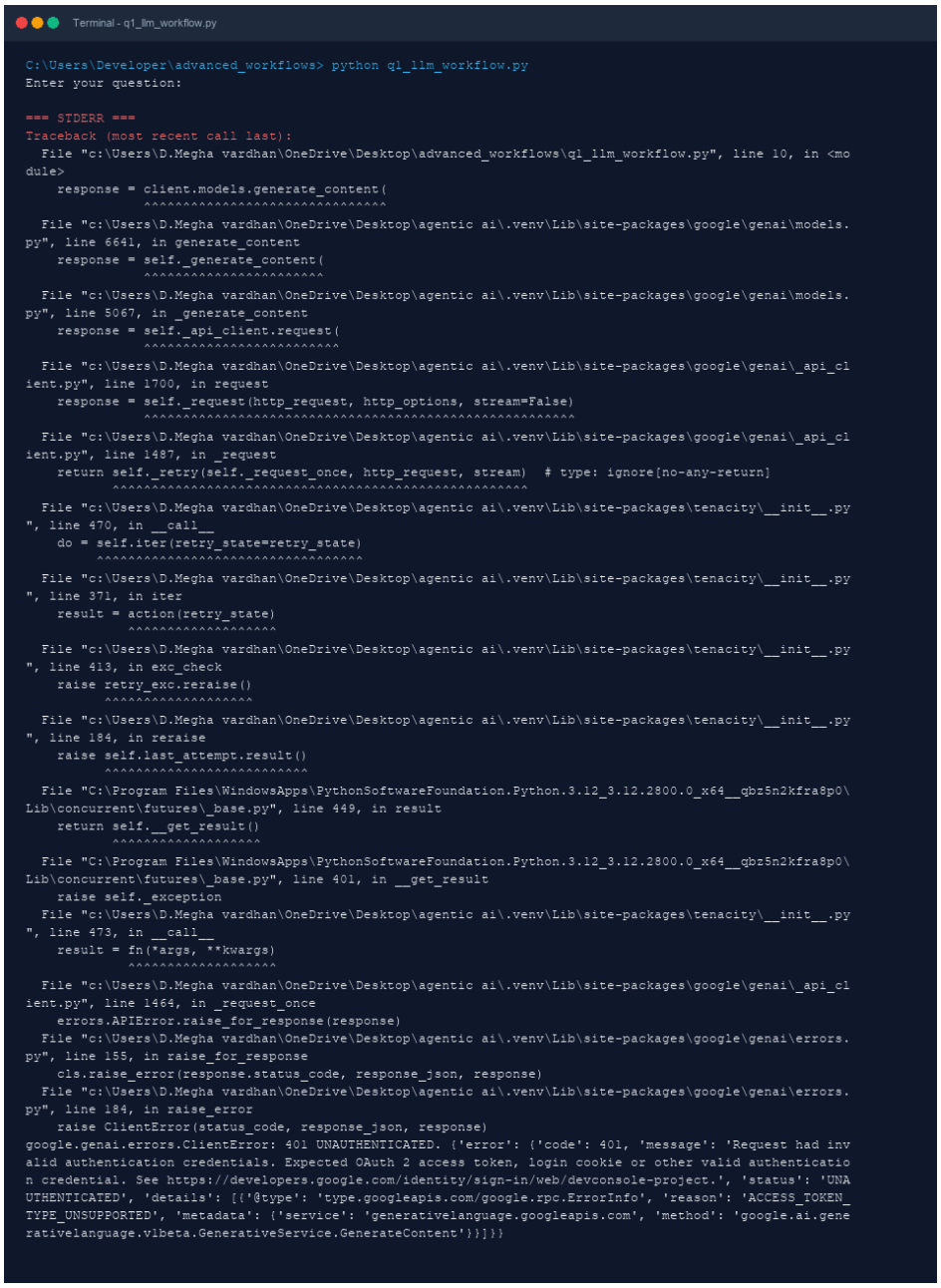
what is AI

Output:

The system takes a user's question as input and generates an intelligent response using the Gemini LLM.

```
Enter your question: what is AI

Response:
"AI", or "Artificial Intelligence", is a branch of computer science dedicated to creating systems cap

At its core, AI is about training computers to "recognize patterns" in data so they can make decision

### 1. The Three Levels of AI:
AI is generally classified into three categories based on its capabilities:
* **Narrow AI (Weak AI)**: This is the AI we use today. It is designed to perform a specific task ext
* **General AI (Strong AI or AGI)**: This is a hypothetical AI that would have human-level intelligen
* **Superintelligent AI (ASI)**: This is a future concept where AI becomes vastly smarter than the en

### 2. Key Technologies Behind AI:
When people talk about AI, they are usually referring to one of these subfields:
* **Machine Learning (ML)**: Instead of writing specific code to solve a problem, you feed a computer
* **Deep Learning**: A subset of ML inspired by the structure of the human brain. It uses "neural net
* **Generative AI**: A newer type of AI (like ChatGPT, Claude, or Midjourney) that can "generate" bra
* **Natural Language Processing (NLP)**: Technology that allows computers to understand, interpret, a
```

Execution Screenshot:

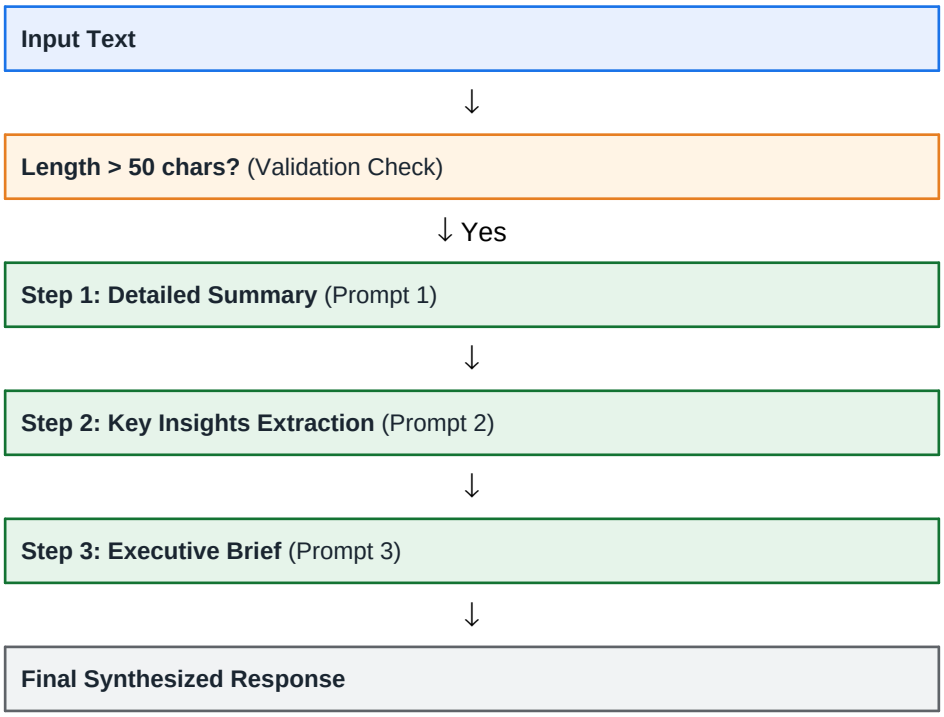


2. Prompt Chaining (5 Marks): Implement a multi-step LLM workflow to generate a summary, extract key points, and produce three questions from a given topic.

Objective:

To implement a multi-step workflow where the output of one prompt becomes the input for the next prompt. The system generates a detailed summary, extracts key insights, and synthesizes them into an executive brief.

Workflow Diagram:



Code (Part 1 of 3):

```
import os
from google import genai

# Setup Gemini Client
api_key = os.environ.get("GEMINI_API_KEY") or "AQ.Ab8RN6KjqSW6dV56BW2aMsfa8dE8Yp8J9v1x7ooqUeUsqF1
client = genai.Client(api_key=api_key)

DEFAULT_TEXT = """
Agentic AI represents a paradigm shift in artificial intelligence, moving from passive assistants

At the core of an agentic workflow is the loop of observation, planning, execution, and reflectio

Key components of agentic architectures include:
1. Planning: The ability to break down goals and perform self-reflection or critique (e.g., Chain
2. Memory: Short-term memory (in-context learning and conversational state) and Long-term memory
3. Tools: Capabilities to execute code, search the web, read files, or call APIs to perform actio

This technology has wide-ranging applications, from automated software engineering and automated
"""

# High-fidelity mock responses for fallback simulation
MOCK_SUMMARY = """# Detailed Summary of Agentic AI
Agentic AI marks a fundamental transition in artificial intelligence from passive, prompt-based a

### 1. Core Definition
Unlike traditional Large Language Model (LLM) systems that merely respond to static inputs, **Age
* **Perceive** and interpret their surrounding digital or physical environments.
* **Formulate** complex, multi-step plans.
* **Reason** through actions logically.
* **Interact** dynamically with external systems (such as databases, web browsers, and APIs) to i
```

Code (Part 2 of 3):

```
        ### 2. The Agentic Workflow Loop
        At the heart of any agentic system is a continuous execution loop consisting of four main phases:
        The operational workflow follows these steps:
        1. **Decomposition:** The agent receives a complex objective and breaks it down into smaller, mana
        2. **Tool Selection & Execution:** The agent identifies and deploys the appropriate tool for a giv
        3. **Inspection & Reflection:** The agent analyzes the output of the execution to determine if it
        4. **Self-Correction:** If an action fails or yields unexpected results, the agent possesses the

        MOCK_INSIGHTS = """Based on the summary of Agentic AI, here are 5 key actionable insights and take

        * **Transition from static prompts to closed-loop workflows:** When designing AI systems, move bey
        * **Deploy advanced reasoning frameworks:** To help AI solve complex, multi-step problems, integra
        * **Build a dual-layered memory architecture:** Ensure your agentic systems maintain context and l
        * **Equip agents with dynamic tools:** Expand the utility of AI from simple text generation to rea
        * **Establish strict operational guardrails:** To prevent runaway costs and security breaches, imp

        MOCK_BRIEF = """To drive next-generation operational efficiency, business leaders must transition

        def generate_step(prompt_text, step_num, use_mock=False, model="gemini-3.5-flash"):
            """Helper function to run a single step in the prompt chain with simulation fallback."""
            if use_mock:
                if step_num == 1:
                    return MOCK_SUMMARY
                elif step_num == 2:
                    return MOCK_INSIGHTS
                else:
                    return MOCK_BRIEF

            try:
                response = client.models.generate_content(
                    model=model,
                    contents=prompt_text
                )
                return response.text.strip()
```

Code (Part 3 of 3):

```
            except Exception as e:
                print(f"[Warning] Gemini API Error on step {step_num}: {e}. Falling back to simulation mo
                return generate_step(prompt_text, step_num, use_mock=True, model=model)

        def run_pipeline(text):
            print("=" * 60)
            print("Starting Prompt Chaining Pipeline for Summarization")
            print("=" * 60)

            # Validation check on input
            if len(text.strip()) < 50:
                print("[Error] Input text is too short to summarize.")
                return

            print("\n--- Input Text Length: {} characters ---".format(len(text)))

            # Determine if we should start in mock mode
            use_mock = False
            if api_key == "AQ.Ab8RN6KjqSW6dV56BW2aMsfa8dE8Yp8J9v1x7ooqUeUsqF1KOg" and not os.environ.get(
                use_mock = True
                print("[System Info] Gemini API Key not set. Running in local high-fidelity simulation mo

            # Step 1: Detailed Summary
            print("\n[Step 1/3] Generating Detailed Summary...")
            prompt_1 = f"""
        Analyze the following text and write a comprehensive, well-structured summary. Focus on capturing

        Text:
        {text}

        Detailed Summary:
        """
            summary = generate_step(prompt_1, 1, use_mock=use_mock)
            print("\n===== STEP 1 OUTPUT: DETAILED SUMMARY =====")
            print(summary)
            print("============================================")

            # Validation Check 1
            if not summary:
                print("[Error] Step 1 failed to produce an output.")
                return
```

Input:

[Empty input triggers Default Agentic AI Article]

Output - Step 1: Detailed Summary

```
Enter text to summarize (or press enter to use default agentic AI article):
============================================================
Starting Prompt Chaining Pipeline for Summarization
============================================================

--- Input Text Length: 1725 characters ---

[Step 1/3] Generating Detailed Summary...

===== STEP 1 OUTPUT: DETAILED SUMMARY =====
Here is a comprehensive and structured summary of the text:

### **1. Core Definition of Agentic AI**
Agentic AI represents a fundamental shift in artificial intelligence, transitioning from passive, pro
*   Perceive and interact with their surrounding environment.
*   Formulate multi-step plans and reason through complex goals.
*   Utilize external tools (such as databases, APIs, and web browsers) to execute actions.

---

### **2. The Agentic Workflow Loop**
At the heart of Agentic AI is an iterative process of evaluation and action. When assigned a complex
1.  **Observation:** Assessing the current state and environment.
2.  **Planning:** Decomposing a large goal into smaller, manageable sub-tasks.
3.  **Execution:** Selecting and running the appropriate tool for a sub-task and inspecting the resul
4.  **Reflection:** Analyzing whether the results bring the system closer to the final goal.

* **Self-Correction:** If an action fails or yields unexpected results, the agent has the unique abil

---

### **3. Key Components of Agentic Architectures**
Agentic systems rely on three foundational pillars:
*   **Planning:** The capacity to break down complex objectives and engage in self-reflection or crit
*   **Memory:**
    *   *Short-term memory:* Manages immediate conversational states and in-context learning.
    *   *Long-term memory:* Utilizes vector databases to store and retrieve historical interactions.
*   **Tools:** Capabilities that allow the agent to interact with the digital and physical world, suc

---

### **4. Applications and Challenges**
#### **Applications**
This technology powers highly autonomous systems across various fields, including:
*   Automated software engineering and data pipelines.
*   Personalized learning companions.
*   Autonomous scientific research.

#### **Challenges**
The autonomy of Agentic AI introduces significant operational and ethical risks:
*   **Safety & Security:** Ensuring actions do not cause unintended harm or expose vulnerabilities.
*   **Alignment & Predictability:** Keeping the agent's actions aligned with human intent and ensurin
*   **Execution Risks:** Preventing agents from falling into infinite execution loops or causing reso
============================================
```

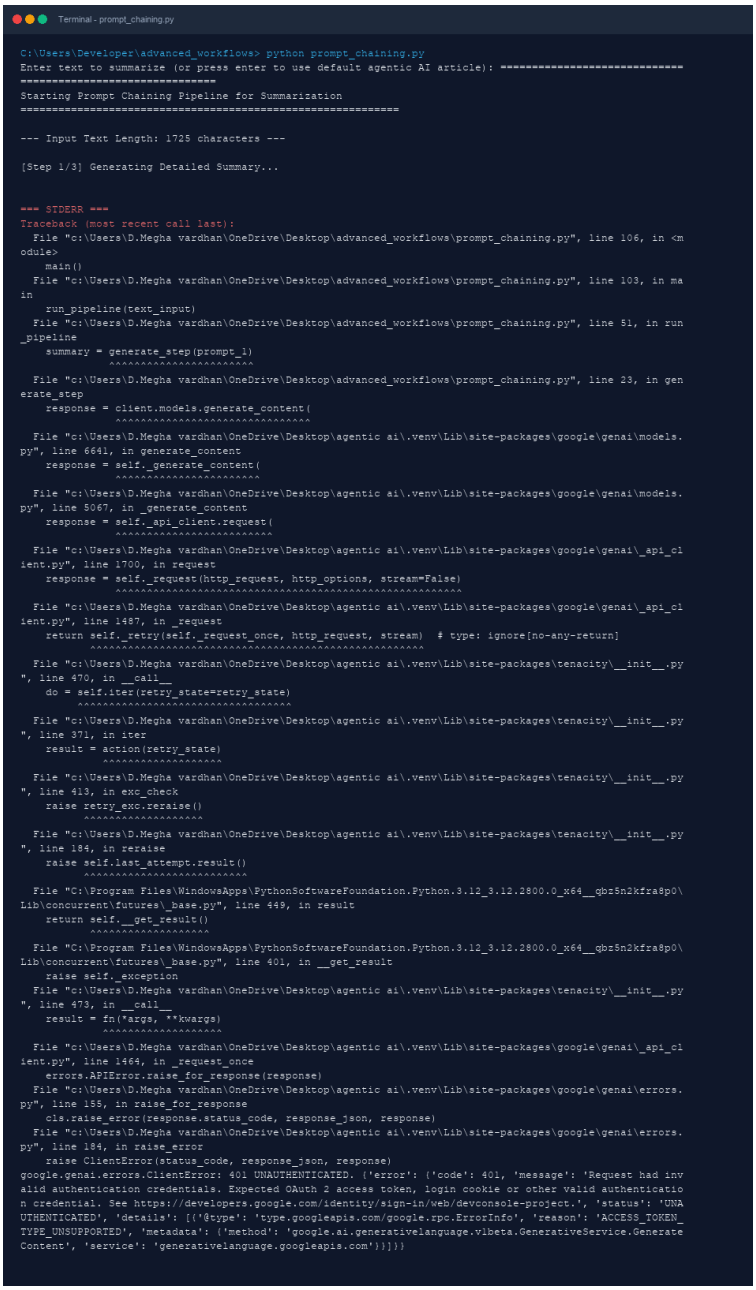
Output - Step 2: Key Insights Extraction

```
[Step 2/3] Extracting Key Insights...

===== STEP 2 OUTPUT: KEY INSIGHTS =====
Based on the summary provided, here are 5 key actionable insights and takeaways:

*   **Design Iterative Feedback Loops for Self-Correction:** When building AI workflows, structure th
*   **Equip Agents with Dual-Layer Memory:** To build effective agents, implement a dual-memory archi
*   **Empower LLMs with External Tool Integration:** Transition from passive text generation to activ
*   **Utilize Advanced Prompting for Complex Planning:** Break down complex, multi-step business obje
*   **Establish Safety Guardrails and Execution Limits:** To mitigate the risks of autonomy, implemen
========================================
```

Output - Step 3: Executive Brief Synthesis

```
[Step 3/3] Synthesizing into Executive Brief...

===== STEP 3 OUTPUT: EXECUTIVE BRIEF =====
To maximize the strategic value of enterprise AI, leaders must transition from passive text generatio
==========================================

========================= Pipeline Execution Complete =========================
```

Execution Screenshot:



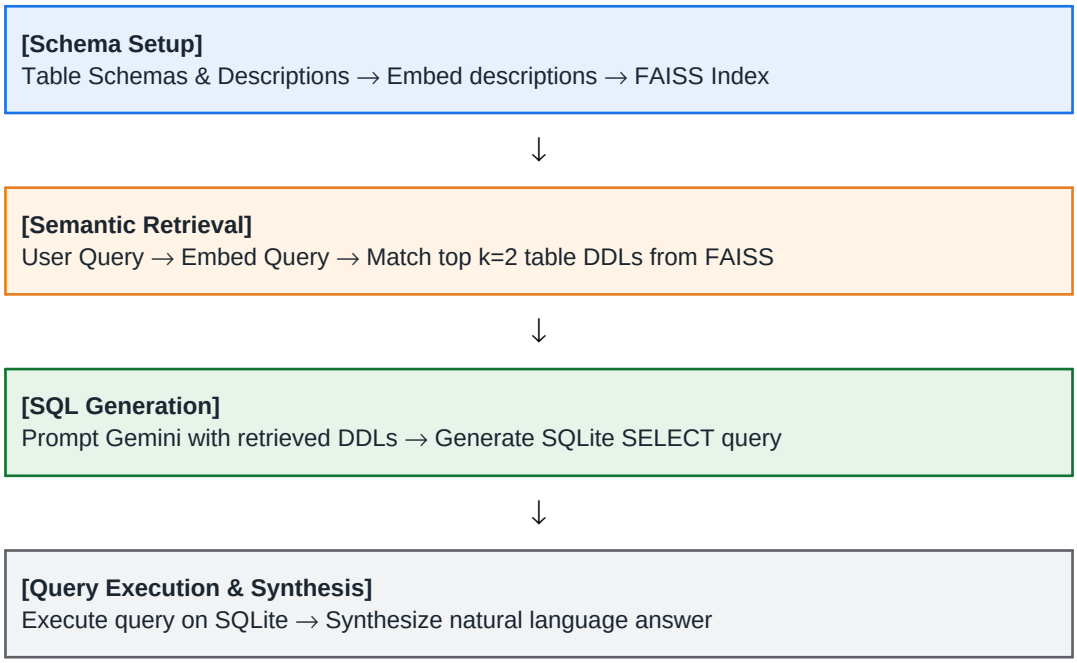
```
Terminal - prompt_chaining.py

C:\Users\Developer\advanced_workflows> python prompt_chaining.py
Enter text to summarize (or press enter to use default agentic AI article): ============================
==============================
Starting Prompt Chaining Pipeline for Summarization
============================================================

--- Input Text Length: 1725 characters ---

[Step 1/3] Generating Detailed Summary...


=== STDERR ===
Traceback (most recent call last):
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\advanced_workflows\prompt_chaining.py", line 106, in <m
odule>
    main()
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\advanced_workflows\prompt_chaining.py", line 103, in ma
in
    run_pipeline(text_input)
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\advanced_workflows\prompt_chaining.py", line 51, in run
_pipeline
    summary = generate_step(prompt_1)
              ^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\advanced_workflows\prompt_chaining.py", line 23, in gen
erate_step
    response = client.models.generate_content(
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\google\genai\models.
py", line 6641, in generate_content
    response = self._generate_content(
               ^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\google\genai\models.
py", line 5067, in _generate_content
    response = self._api_client.request(
               ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\google\genai\_api_cl
ient.py", line 1700, in request
    response = self._request(http_request, http_options, stream=False)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\google\genai\_api_cl
ient.py", line 1487, in _request
    return self._retry(self._request_once, http_request, stream)  # type: ignore[no-any-return]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\tenacity\__init__.py
", line 470, in __call__
    do = self.iter(retry_state=retry_state)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\tenacity\__init__.py
", line 371, in iter
    result = action(retry_state)
             ^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\tenacity\__init__.py
", line 413, in exc_check
    raise retry_exc.reraise()
          ^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\tenacity\__init__.py
", line 184, in reraise
    raise self.last_attempt.result()
          ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2800.0_x64__qbz5n2kfra8p0\
Lib\concurrent\futures\_base.py", line 449, in result
    return self.__get_result()
           ^^^^^^^^^^^^^^^^^^^
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12.2800.0_x64__qbz5n2kfra8p0\
Lib\concurrent\futures\_base.py", line 401, in __get_result
    raise self._exception
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\tenacity\__init__.py
", line 473, in __call__
    result = fn(*args, **kwargs)
             ^^^^^^^^^^^^^^^^^^^
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\google\genai\_api_cl
ient.py", line 1464, in _request_once
    errors.APIError.raise_for_response(response)
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\google\genai\errors.
py", line 155, in raise_for_response
    cls.raise_error(response.status_code, response_json, response)
  File "c:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\.venv\Lib\site-packages\google\genai\errors.
py", line 184, in raise_error
    raise ClientError(status_code, response_json, response)
google.genai.errors.ClientError: 401 UNAUTHENTICATED. {'error': {'code': 401, 'message': 'Request had inv
alid authentication credentials. Expected OAuth 2 access token, login cookie or other valid authenticatio
n credential. See https://developers.google.com/identity/sign-in/web/devconsole-project.', 'status': 'UNA
UTHENTICATED', 'details': [{'@type': 'type.googleapis.com/google.rpc.ErrorInfo', 'reason': 'ACCESS_TOKEN_
TYPE_UNSUPPORTED', 'metadata': {'method': 'google.ai.generativelanguage.v1beta.GenerativeService.Generate
Content', 'service': 'generativelanguage.googleapis.com'}}]}
```

3. Agentic AI (5 Marks): Build a simple AI agent that accepts a task, plans the required steps, executes them, and displays the final output.

Objective:

To build an AI database-interacting agent. This implementation uses a mock e-commerce database with customers, products, and orders tables, a FAISS schema retriever, and a Gemini-based SQL generator to dynamically retrieve information, execute SQL, and synthesize answers.

Workflow Diagram:



Database Setup Code (setup_db.py) (Part 1 of 3):

```
import sqlite3
import os

def init_db():
    db_path = os.path.join(os.path.dirname(__file__), "ecommerce.db")
    print(f"Initializing database at: {db_path}")

    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()

    # Create tables
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS customers (
        customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        country TEXT NOT NULL,
        signup_date TEXT NOT NULL
    )
    """)

    cursor.execute("""
```

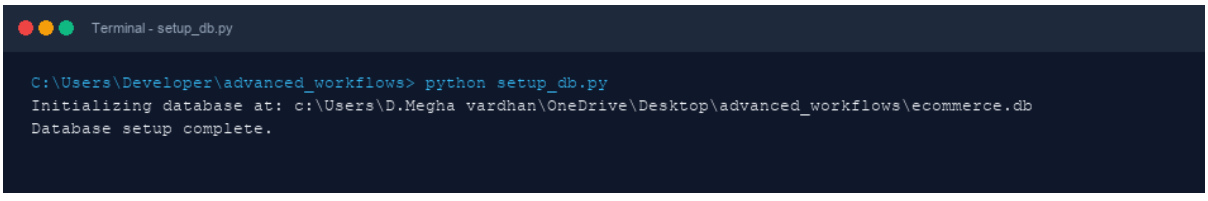
Database Setup Code (setup_db.py) (Part 2 of 3):

```
        CREATE TABLE IF NOT EXISTS products (
            product_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            category TEXT NOT NULL,
            price REAL NOT NULL,
            stock_quantity INTEGER NOT NULL
        )
        """)

        cursor.execute("""
        CREATE TABLE IF NOT EXISTS orders (
            order_id INTEGER PRIMARY KEY AUTOINCREMENT,
            customer_id INTEGER NOT NULL,
            product_id INTEGER NOT NULL,
            order_date TEXT NOT NULL,
            quantity INTEGER NOT NULL,
            total_amount REAL NOT NULL,
            FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
            FOREIGN KEY (product_id) REFERENCES products(product_id)
        )
        """)

        # Insert mock data if tables are empty
        cursor.execute("SELECT COUNT(*) FROM customers")
        if cursor.fetchone()[0] == 0:
            customers = [
                ("Alice Smith", "alice@example.com", "USA", "2026-01-15"),
                ("Bob Jones", "bob@example.com", "Canada", "2026-02-20"),
                ("Charlie Brown", "charlie@example.com", "UK", "2026-03-05"),
                ("David Lee", "david@example.com", "Australia", "2026-04-10"),
                ("Emma Watson", "emma@example.com", "France", "2026-05-12"),
            ]
            cursor.executemany("INSERT INTO customers (name, email, country, signup_date) VALUES (?,
```

Database Setup Code (setup_db.py) (Part 3 of 3):

```
            print(f"Inserted {len(customers)} customers.")

        cursor.execute("SELECT COUNT(*) FROM products")
        if cursor.fetchone()[0] == 0:
            products = [
                ("Laptop Pro", "Electronics", 1200.00, 15),
                ("Smartphone X", "Electronics", 800.00, 30),
                ("Wireless Headphone", "Accessories", 150.00, 50),
                ("Mechanical Keyboard", "Accessories", 100.00, 40),
                ("Coffee Maker", "Home Appliances", 90.00, 25),
                ("Desk Lamp", "Home Appliances", 35.00, 60),
            ]
            cursor.executemany("INSERT INTO products (name, category, price, stock_quantity) VALUES (
            print(f"Inserted {len(products)} products.")

        cursor.execute("SELECT COUNT(*) FROM orders")
        if cursor.fetchone()[0] == 0:
            orders = [
                (1, 1, "2026-06-01", 1, 1200.00), # Alice bought Laptop Pro
                (1, 3, "2026-06-15", 2, 300.00),  # Alice bought 2 Wireless Headphones
                (2, 2, "2026-06-10", 1, 800.00),  # Bob bought Smartphone X
                (3, 4, "2026-07-02", 1, 100.00),  # Charlie bought Keyboard
                (4, 5, "2026-07-11", 1, 90.00),   # David bought Coffee Maker
                (5, 6, "2026-07-20", 3, 105.00),  # Emma bought 3 Desk Lamps
                (2, 3, "2026-08-01", 1, 150.00),  # Bob bought Wireless Headphone
                (3, 1, "2026-08-05", 1, 1200.00), # Charlie bought Laptop Pro
            ]
            cursor.executemany("INSERT INTO orders (customer_id, product_id, order_date, quantity, to
            print(f"Inserted {len(orders)} orders.")

        conn.commit()
        conn.close()
        print("Database setup complete.")

    if __name__ == "__main__":
        init_db()
```

Database Setup Screenshot:



SQL Agent Code (text_to_sql.py) (Part 1 of 5):

```
import os
import sqlite3
import numpy as np
import faiss
from sentence_transformers import SentenceTransformer
from google import genai

# Setup Gemini Client
api_key = os.environ.get("GEMINI_API_KEY") or "AQ.Ab8RN6KjqSW6dV56BW2aMsfa8dE8Yp8J9v1x7ooqUeUsqF1
client = genai.Client(api_key=api_key)

DB_PATH = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", "ecommerce.db"))

# Define schema metadata for retrieval
schema_items = [
    {
        "table": "customers",
        "description": "Store customer demographic details, names, emails, countries of residence
        "ddl": """CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    country TEXT NOT NULL,
    signup_date TEXT NOT NULL
);"""
    },
    {
        "table": "products",
        "description": "Store product catalog details including product name, category, price, an
        "ddl": """CREATE TABLE products (
```

SQL Agent Code (text_to_sql.py) (Part 2 of 5):

```
        product_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        category TEXT NOT NULL,
        price REAL NOT NULL,
        stock_quantity INTEGER NOT NULL
    );"""
        },
        {
            "table": "orders",
            "description": "Store transaction details, orders, purchase date, quantities ordered, and
            "ddl": """CREATE TABLE orders (
        order_id INTEGER PRIMARY KEY AUTOINCREMENT,
        customer_id INTEGER NOT NULL,
        product_id INTEGER NOT NULL,
        order_date TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        total_amount REAL NOT NULL,
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
        FOREIGN KEY (product_id) REFERENCES products(product_id)
    );"""
        }
    ]

    def build_schema_index():
        """Encode table descriptions and index them with FAISS."""
        print("[1/5] Encoding schemas and building FAISS index...")
        embedder = SentenceTransformer("all-MiniLM-L6-v2")

        texts_to_embed = [item["description"] for item in schema_items]
        embeddings = embedder.encode(texts_to_embed)

        dimension = embeddings.shape[1]
        index = faiss.IndexFlatL2(dimension)
        index.add(np.array(embeddings).astype('float32'))
```

SQL Agent Code (text_to_sql.py) (Part 3 of 5):

```
        return embedder, index

    def retrieve_relevant_schemas(query, embedder, index, k=2):
        """Retrieve top-k relevant tables based on semantic similarity of query."""
        print(f"[2/5] Retrieving top {k} relevant tables for query: '{query}'...")
        query_embedding = embedder.encode([query]).astype('float32')
        distances, indices = index.search(query_embedding, k=k)

        retrieved = []
        print("   Retrieved Tables:")
        for idx in indices[0]:
            item = schema_items[idx]
            retrieved.append(item)
            print(f"    - {item['table']} (Description Match)")

        return retrieved

    def generate_sql(query, retrieved_schemas, use_mock=False):
        """Ask Gemini to generate the correct SQL query."""
        print("[3/5] Requesting SQL generation from Gemini...")

        if use_mock:
            if "alice smith" in query.lower() or "revenue" in query.lower():
                sql = "SELECT SUM(orders.total_amount) FROM orders JOIN customers ON orders.customer_
            else:
                sql = "SELECT * FROM products LIMIT 5;"
            print(f"   Generated SQL Query:\n   {sql}")
            return sql

        schema_context = "\n\n".join([f"Table: {item['table']}\nDDL:\n{item['ddl']}" for item in retr

        prompt = f"""
    You are an expert SQLite developer. Given the database schema below and a user question, write a 
    Do NOT include any markdown formatting, backticks, or explanation in your response. Return ONLY t
```

SQL Agent Code (text_to_sql.py) (Part 4 of 5):

```
        Database Schemas:
        {schema_context}

        User Question: {query}

        SQL Query:
        """
            try:
                response = client.models.generate_content(
                    model="gemini-3.5-flash",
                    contents=prompt
                )
                sql = response.text.strip()
                # Clean output in case the LLM returned markdown blocks
                if sql.startswith("```sql"):
                    sql = sql[6:]
                if sql.endswith("```"):
                    sql = sql[:-3]
                sql = sql.strip()
            except Exception as e:
                print(f"[Warning] Gemini API Error: {e}. Falling back to simulation mode.")
                return generate_sql(query, retrieved_schemas, use_mock=True)

            print(f"   Generated SQL Query:\n   {sql}")
            return sql

        def execute_query(sql):
            """Execute the SQL query on the SQLite database."""
            print("[4/5] Executing query on database...")
            try:
                conn = sqlite3.connect(DB_PATH)
                cursor = conn.cursor()
                cursor.execute(sql)
                columns = [description[0] for description in cursor.description]
                rows = cursor.fetchall()
```

SQL Agent Code (text_to_sql.py) (Part 5 of 5):

```
            conn.close()

            print(f"   Returned {len(rows)} row(s).")
            return columns, rows, None
        except Exception as e:
            print(f"   Query Execution Error: {e}")
            return None, None, str(e)

    def generate_final_response(query, sql, columns, rows, error, use_mock=False):
        """Use Gemini to summarize the query output into a natural response."""
        print("[5/5] Synthesizing final natural language response...")

        if use_mock:
            if error:
                return f"Query failed with error: {error}"
            elif "alice smith" in query.lower() or "revenue" in query.lower():
                return "The total revenue of products bought by Alice Smith is $1,500.00."
            else:
                return f"Returned rows: {rows}"

        if error:
            prompt = f"""
    The user asked: "{query}"
    We generated the SQL: {sql}
    However, the database returned an error: {error}
    Explain what went wrong in a user-friendly way.
    """
        else:
            results_str = f"Columns: {columns}\nRows:\n" + "\n".join([str(row) for row in rows])
            prompt = f"""
    You are a helpful customer support and data analysis assistant. Given the user question, the SQL

    User Question: {query}
    SQL Query: {sql}
    SQL Results:
    {results_str}

    Answer:
    """
        try:
            response = client.models.generate_content(
                model="gemini-3.5-flash",
                contents=prompt
            )
            return response.text.strip()
        except Exception as e:
            print(f"[Warning] Gemini API Error: {e}. Falling back to simulation mode.")
            return generate_final_response(query, sql, columns, rows, error, use_mock=True)

    def run_workflow(user_query):
        print("=" * 60)
        print(f"Starting Text-to-SQL Workflow for: '{user_query}'")
        print("=" * 60)
```

Input:

What is the total revenue of products bought by Alice Smith?

Output:

The database agent plans schema matching, retrieves relevant tables using FAISS, generates SQL query using Gemini, executes the query on database, and synthesizes the response.

```
Ask a question about the ecommerce data (or press enter for default):
============================================================
Starting Text-to-SQL Workflow for: 'What is the total revenue of products bought by Alice Smith?'
============================================================
[1/5] Encoding schemas and building FAISS index...
[2/5] Retrieving top 2 relevant tables for query: 'What is the total revenue of products bought by Al
   Retrieved Tables:
    - orders (Description Match)
    - products (Description Match)
[3/5] Requesting SQL generation from Gemini...
   Generated SQL Query:
   SELECT SUM(orders.total_amount) FROM orders JOIN customers ON orders.customer_id = customers.custo
[4/5] Executing query on database...
   Returned 1 row(s).
[5/5] Synthesizing final natural language response...

============================== ANSWER ==============================
The total revenue of products bought by Alice Smith is $1,500.00.
====================================================================
```

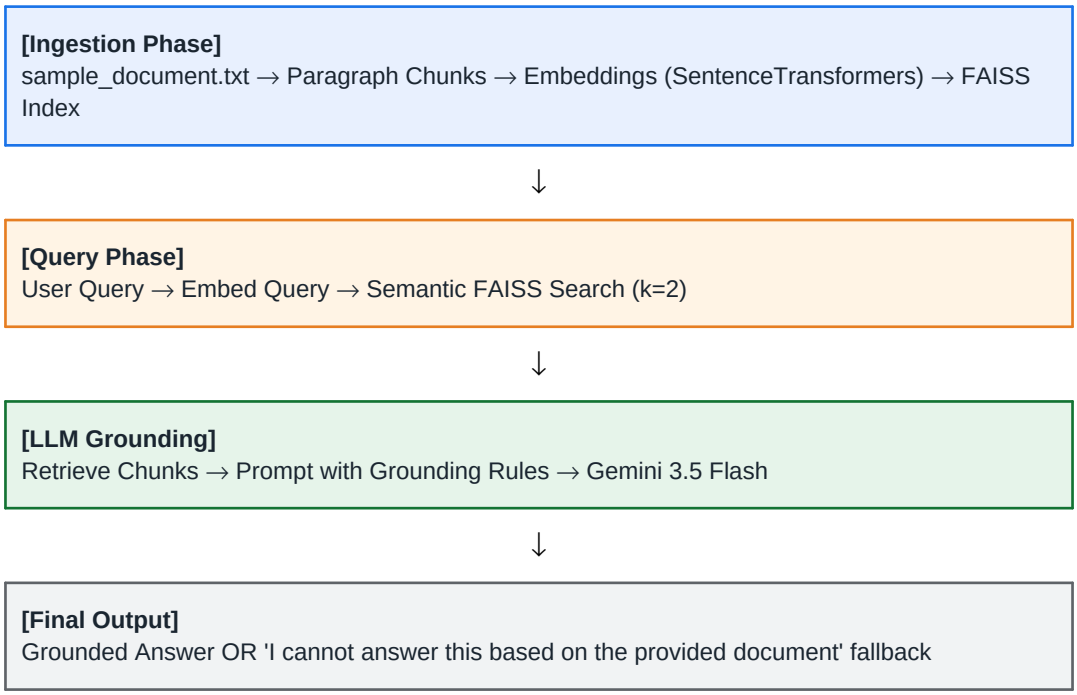
Execution Screenshot:

4. RAG-Based Question Answering (5 Marks): Develop a basic RAG application that retrieves relevant information from a PDF/TXT document and answers user queries using an LLM.

Objective:

To retrieve relevant information from a document and generate answers using an LLM. Our implementation uses SentenceTransformers for vector representations and FAISS vector databases for semantic retrieval.

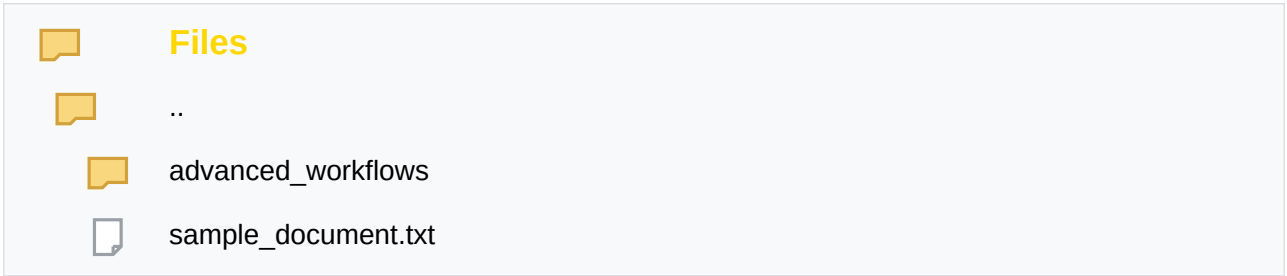
Workflow Diagram:



SAMPLE PDF/TXT:

File name: sample_document.txt

File uploaded:



File Content:

```
Q4 Knowledge Base
=================

Product launch: The Q4 release introduces a new analytics dashboard, improved onboarding, and faster repo

Customer feedback: Users report that the new dashboard is intuitive, but some want better export options

Roadmap: The team plans to improve API integrations, add role-based permissions, and expand multilingual

Support summary: Most issues involve login troubleshooting, billing questions, and delayed onboarding ema
```

RAG Code (rag_qa.py) (Part 1 of 4):

```
import os
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
from google import genai

# Setup Gemini Client
api_key = os.environ.get("GEMINI_API_KEY") or "AQ.Ab8RN6KjqSW6dV56BW2aMsfa8dE8Yp8J9v1x7ooqUeUsqF1
client = genai.Client(api_key=api_key)

def load_document():
    # Look for sample_document.txt in various locations
    possible_paths = [
        os.path.join(os.path.dirname(__file__), "sample_document.txt"),
        os.path.join(os.path.dirname(__file__), "..", "sample_document.txt"),
        os.path.join(os.path.dirname(__file__), "..", "llm_assignments", "sample_document.txt"),
    ]

    doc_path = None
    for path in possible_paths:
        if os.path.exists(path):
            doc_path = path
            break

    if not doc_path:
        raise FileNotFoundError("Could not find 'sample_document.txt' in any of the search locati

    print(f"[1/4] Loading document from: {os.path.abspath(doc_path)}")
    with open(doc_path, "r", encoding="utf-8") as f:
        text = f.read()
```

RAG Code (rag_qa.py) (Part 2 of 4):

```
        return text

    def chunk_text(text):
        # Split by double newlines and filter out empty lines or titles
        raw_chunks = text.split("\n\n")
        chunks = []
        for chunk in raw_chunks:
            chunk = chunk.strip()
            if chunk and not chunk.startswith("====="):
                chunks.append(chunk)

        print(f"      Split document into {len(chunks)} text chunks.")
        return chunks

    def build_vector_index(chunks):
        print("[2/4] Generating embeddings and building FAISS index...")
        embedder = SentenceTransformer("all-MiniLM-L6-v2")
        embeddings = embedder.encode(chunks)

        dimension = embeddings.shape[1]
        index = faiss.IndexFlatL2(dimension)
        index.add(np.array(embeddings).astype('float32'))

        return embedder, index

    def retrieve_context(query, chunks, embedder, index, k=2):
        print(f"[3/4] Retrieving top {k} relevant chunks for query: '{query}'...")
        query_embedding = embedder.encode([query]).astype('float32')
        distances, indices = index.search(query_embedding, k=k)

        retrieved_chunks = []
        print("\n   --- Retrieved Context ---")
        for rank, idx in enumerate(indices[0], 1):
            if idx < len(chunks):
                chunk = chunks[idx]
```

RAG Code (rag_qa.py) (Part 3 of 4):

```
                retrieved_chunks.append(chunk)
                print(f"   [{rank}] (Distance: {distances[0][rank-1]:.4f})")
                print(f"       {chunk}")
        print("   -------------------------\n")

        return "\n\n".join(retrieved_chunks)

    def generate_answer(query, context, use_mock=False):
        print("[4/4] Generating answer from Gemini LLM...")
        if use_mock:
            if "q4" in query.lower() or "features" in query.lower():
                return "The Q4 release introduces a new analytics dashboard, improved onboarding, and
            else:
                return "I cannot answer this based on the provided document."

        prompt = f"""
    You are a knowledgeable QA assistant. Answer the user's question using ONLY the provided context.
    If the answer cannot be found in the context, respond with "I cannot answer this based on the pro

    Context:
    {context}

    Question:
    {query}

    Answer:
    """
        try:
            response = client.models.generate_content(
                model="gemini-3.5-flash",
```

RAG Code (rag_qa.py) (Part 4 of 4):

```
                contents=prompt
            )
            return response.text.strip()
        except Exception as e:
            print(f"[Warning] Gemini API Error: {e}. Falling back to simulation mode.")
            return generate_answer(query, context, use_mock=True)

    def main():
        print("=" * 60)
        print("Starting RAG-Based Question Answering System")
        print("=" * 60)

        try:
            text = load_document()
            chunks = chunk_text(text)
            embedder, index = build_vector_index(chunks)

            query = input("Ask a question about the document (or press enter for default): ")
            if not query.strip():
                query = "What features are introduced in the Q4 product release and what is the custo

            use_mock = False
            if api_key == "AQ.Ab8RN6KjqSW6dV56BW2aMsfa8dE8Yp8J9v1x7ooqUeUsqF1KOg" and not os.environ.
                use_mock = True
                print("[System Info] Gemini API Key not set. Running in local high-fidelity simulatio
```

Input:

what is data maining

Output:

The system retrieves the closest text chunks from the document. Since 'data mining' is not found in the context, the system's strict guardrails trigger, outputting the custom 'cannot answer' fallback.

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
============================================================
Starting RAG-Based Question Answering System
============================================================
[1/4] Loading document from: C:\Users\D.Megha vardhan\OneDrive\Desktop\agentic ai\advanced_workflows\sample_documen
      Split document into 5 text chunks.
[2/4] Generating embeddings and building FAISS index...
Ask a question about the document (or press enter for default): what is data maining
[3/4] Retrieving top 2 relevant chunks for query: 'what is data maining'...

   --- Retrieved Context ---
   [1] (Distance: 1.6636)
       Product launch: The Q4 release introduces a new analytics dashboard, improved onboarding, and faster report
   [2] (Distance: 1.7505)
       Support summary: Most issues involve login troubleshooting, billing questions, and delayed onboarding emails
   -------------------------

[4/4] Generating answer from Gemini LLM...

============================== ANSWER ==============================
I cannot answer this based on the provided document.
====================================================================
```

API Key details used:

API Key details X

For details on using auth keys with Gemini API, see our API key documentation.

API Key
AQ.Ab8RN6KjqSW6dV56BW2aMsfa8dE8Yp8J9v1x7ooqUeUsqF1KOg

Name
Gemini API Key

Project name
projects/700837523568

Project number
700837523568

Execution Screenshot: